

Yazılım Bakımı: Sistem Bilgisinin Biriktirilmesi ve Ölçülmesi

ÖFY

August 2026

Özet

Bir yazılım üzerinde çalışıldıkça biriken şey yalnızca kod değil, sistem hakkındaki kanıtlanmış bilgidir; testler bu bilginin kod tabanına gömülmüş hâlidir. Test türleri birbirinden yalnızca teknik olarak değil, taşıdıkları bilgi türü bakımından ayrılır: davranış testi mevcut sözleşmeyi, regresyon testi geçmişte yaşanmış bir kırılmayı, entegrasyon testi bileşenler arası ilişkiyi, sınır testi ise geçerlilik alanının nerede bittiğini kaydeder. Bu dörtlüden türetilen sistem anlama boyutları literatürde birleşik bir taksonomi olarak bulunmasa da, parçaları program anlama, sistem mühendisliği ve yazılım evrimi alanlarında bağımsız olarak yerleşiktir; ayrıca Zachman çerçevesi ve mimari görünüm modelleri, aynı ayrımı otuz yılı aşkın süredir standartlaştırılmış biçimde içerir. “Neden böyle?” sorusunun kaydı, tasarım gerekçesi (*design rationale*) adıyla ayrı bir literatüre sahiptir ve kodda saklanamayan tek bilgi türüdür. Test kapsamı, anlaşılmanın ölçüsü değildir: kodun çalıştırıldığını gösterir, denetlendiğini göstermez; bu ayrımın ölçülebilir karşılığı mutasyon testidir. Yerel bir düzeltmenin uzaktaki bir davranışı bozması, değişiklik etki analizinin konusudur; ancak etki analizi bir tahmindir ve tahminin kaçırıldığını yakalayan mekanizma, konuma bağlı olmayan değişmez denetimidir.

Giriş

Bir yazılım projesinin başlangıcında sistem hakkında bilinen şey azdır. Kod yazıldıkça, hata bulundukça, sınır durumlar keşfedildikçe ve üretim ortamında gözlem yapıldıkça bu bilgi artar. Artan bilginin bir kısmı kodun kendisinde, bir kısmı testlerde, bir kısmı ise hiçbir yerde saklanmaz ve ekip değiştiğinde kaybolur.

Bu durumu tarif eden bir benzetme, yazılımı beyaz bir sayfa olarak düşünmektir. Sayfaya bırakılan her iz — yazılan test, düzeltilen hata, çıkarılan bağımlılık haritası — sistem hakkında edinilmiş bir bilginin kayıdır. Benzetmenin ters çevirdiği sezgi şudur: sayfanın kararması bir kirlenme değil, belirsizliğin azalmasıdır. Çok sayıda test içeren bir kod tabanı ilk bakışta karmaşık görünür; taşıdığı asıl anlam, sistemin sınırlarının haritalanmış olmasıdır.

Bu çerçeve iki soru doğurur. Birincisi, bu birikmiş bilginin hangi türlere ayrıldığıdır. İkincisi, ne kadarının biriktiğinin ölçülüp ölçülemeyeceğidir.

Notun kullandığı terimler baştan ayrılmalıdır. **Program anlama** (*program comprehension*), bir geliştiricinin mevcut bir yazılım hakkında zihinsel model kurma sürecidir; alan, von Mayrhauser ve Vans'ın (1995) derlemesiyle sistematik hâle gelmiştir. **Yazılım evrimi** (*software evolution*), yazılımın zaman içindeki değişimini inceleyen alandır; Lehman'ın (1980) değişim yasaları bu alanın kurucu metinlerindendir. **Değişiklik etki analizi** (*change impact analysis*), bir değişikliğin sistemin başka hangi kısımlarını etkileyebileceğinin önceden belirlenmesidir.

Notun sınırı, sistem hakkındaki bilginin türleri, ölçülmesi ve bir değişiklik sırasında korunmasıdır. Test tekniklerinin kendisi ve araç seçimi kapsam dışıdır.

Testlerin taşıdığı bilgi türleri

Bir test yalnızca “kod doğru mu” sorusuna cevap vermez. Test türleri arasındaki fark, arkalarındaki sorunun farkıdır.

Dört soru

- **Davranış testi.** “Sistemin şu davranışı vardır ve korunması istenmektedir.” Kaydedilen bilgi, sistemin şu andaki sözleşmesidir.
- **Regresyon testi.** “Burada geçmişte bir kırılma yaşandı ve tekrarına izin verilmemektedir.” Kaydedilen bilgi, sistemin hafızasıdır. Test burada yalnızca doğrulama yapmaz; gerçekleşmiş bir olayın bilgisini kod tabanına gömer.
- **Entegrasyon testi.** “Bu iki bileşenin birlikte nasıl davranması gerektiği bilinmektedir.” Kaydedilen bilgi, sistemin ilişki yapısıdır.
- **Sınır testi.** “Sistemin geçerlilik alanının nerede bittiği keşfedilmiştir.” Kaydedilen bilgi, davranış uzayının sınırlarıdır.

Bu dördünün zaman yönleri de farklıdır. Davranış testi şimdiye, regresyon testi geçmişe, entegrasyon testi yapıya, sınır testi ise geçerlilik alanına bakar.

Test kümesinin ikinci işlevi

Bu ayırım yapıldığında, test kümesinin kalite denetimi dışında bir işlevi daha görünür hâle gelir: sistemin davranışsal kaydı olmak. Olgun bir test kümesine bakan bir geliştirici yalnızca kodun çalıştığını değil, sistemin ne yapması gerektiğini, geçmişte nerelerde kırıldığını, bileşenlerin nasıl bağlandığını ve sınırların nerede olduğunu okuyabilir.

Bu, program anlama literatürünün merkezinde duran gözlemle uyumludur: geliştirici kodu okurken kodun kendisini değil, kodun temsil ettiği sistemi yeniden kurar (von Mayrhauser ve Vans, 1995).

Sistem anlamının boyutları

Testlerden türetilen beşli

Yukarıdaki dört testin taşıdığı bilgi, test türü olmaktan çıkarılıp soyutlandığında dört boyut elde edilir:

- **Davranış** (*behavior*) — sistem ne yapıyor.
- **Yapı** (*structure*) — parçalar nasıl organize olmuş ve nasıl bağlı.
- **Evrim** (*evolution*) — sistem bu hâle nasıl geldi.
- **Sınır** (*boundary*) — sistem nerede ve hangi koşullarda geçerli.

Bu dördü, bir sistem hakkında bilinmesi gerekenlerin tamamını kapsamaz. Eksik kalan bilgi türü şu örnekle görünür hâle gelir:

```
if (user.purchaseCount > 10) {  
    return true;  
}
```

Davranış bilinmektedir: on satın almadan sonra kullanıcı ayrıcalıklı sayılmaktadır. Yapı bilinmektedir: kural hangi bileşende ve hangi çağrı zincirinde yer almaktadır. Evrim bilinmektedir: eşik önce beş iken sonradan ona çıkarılmıştır. Sınır bilinmektedir: kural yalnızca kayıtlı kullanıcılara uygulanmaktadır. Buna rağmen kritik bir bilgi eksiktir: eşik neden on olduğu. Sayı teknik bir zorunluluk değil, bir iş kararının teknik karşılığıdır. Bu bilgi olmadan bir geliştirici eşik beşe indirebilir; kod çalışır, testler geçer, mimari bozulmaz, ancak kararın kendisi bozulur.

Bu, beşinci boyutu getirir: **amaç** (*purpose*) — sistem bunu neden yapıyor.

Amaç ile evrimin ayrımı

İki boyut sık karıştırılır. Evrim “bu değişiklik ne zaman ve nasıl oldu” sorusunun kayıdır. Amaç “bu davranışın gerekçesi nedir” sorusunun kayıdır. “Önbellek 2024’te kaldırıldı” bir evrim bilgisidir; “çünkü yetki değişikliklerinin anında görünmesi gerekiyordu” bir amaç bilgisidir. Birincisi sürüm geçmişinden çıkarılabilir, ikincisi çıkarılamaz.

Literatürdeki karşılıklar

Bu beşli, tek bir alanın standart taksonomisi değildir; ancak parçalarının her birinin bağımsız karşılığı vardır.

Alanlara dağılmış karşılıklar

Program anlama literatüründe yapı ve işlev, yüksek seviyeli anlamının iki merkezi bilgi türü olarak ele alınır. Brooks'un (1983) yukarıdan aşağı anlama modelinde geliştirici işe programın amacına dair bir hipotez kurarak başlar ve kodu bu hipotezi doğrulamak için okur; amaç bilgisinin süreç içindeki önceliği bu modelde açıkça yer alır.

Sistem mühendisliği tarafında sınır ve bağlam yerleşik kavramlardır. Sistem sınırı, ilgi konusu sistemi dış varlıklardan ayırır; bağlam diyagramı, sistem ile çevresi arasındaki girdi, çıktı ve etkileşimleri gösterir.

Yazılım evrimi ise başlı başına bir alandır. Lehman'ın (1980) yasaları, kullanımda olan bir yazılımın değişmek zorunda olduğunu ve değişmedikçe kullanışlılığını yitirdiğini ortaya koyar; sistemin bugünkü hâlinin yalnızca mevcut gereksinimlerle açıklanamaz oluşunun kaynağı budur.

Halihazırda standartlaşmış ayrımlar

Bu beşlinin literatürde karşılığı bulunmadığı sonucuna varmak yanlış olur. En yakın iki karşılık şunlardır.

Birincisi Zachman'ın (1987) bilgi sistemleri mimarisi çerçevesidir. Çerçeve, bir sistemin tanımını altı soru ekseninde düzenler: ne, nasıl, nerede, kim, ne zaman ve neden. “Neden” eksenini burada baştan ayrı bir sütundur; amacın ayrı bir boyut olup olmadığı tartışması, bu çerçevede otuz yılı aşkın süredir ayrılmış hâldedir.

İkincisi mimari görünüm modelleridir. Kruchten'in (1995) 4+1 modeli sistemi mantıksal, süreç, geliştirme ve fiziksel görünümle, bunları bağlayan senaryolarla tanımlar. ISO/IEC/IEEE 42010 standardı bu yaklaşımı genelleştirir: bir mimari tanım tek bir açıklamadan değil, farklı paydaş ilgilerine karşılık gelen birden fazla görünümünden oluşur. “Neden birden fazla boyut gereklidir” sorusunun standartlaşmış cevabı budur ve cevap, boyut sayısının sabit olmadığı, paydaş ilgilerine göre belirlendiği yönündedir.

Amacın literatürdeki adı

“Neden böyle?” bilgisinin yerleşik adı **tasarım gerekçesidir** (*design rationale*). Kavramın kökeni Kunz ve Rittel'in (1970) IBIS yaklaşımına dayanır: bir tasarım kararı, çözülen sorun, değerlendirilen konumlar ve bunların lehine ve aleyhine öne sürülen argümanlarla birlikte kaydedilir. Parnas ve Clements (1986), gerçek tasarım sürecinin rasyonel olmadığını kabul etmekle birlikte, sonucun rasyonel bir süreçten çıkmış gibi belgelenmesini savunur; gerekçe kaydının işlevi budur.

Güncel pratikteki karşılığı mimari karar kayıdır (*architecture decision record*). Kayıt, kararın ne olduğunu değil hangi seçenekler arasından, hangi gerekçeyle ve hangi sonuçlar göze alınarak seçildiğini içerir (Nygard, 2011).

Kavramın önemi şu gözlemden gelir: gerekçe kodda saklanamaz. Kod, kararın sonucunu taşır, gerekçesini taşımaz. // DO NOT CACHE biçiminde bir uyarı tek başına yetersizdir; neden önbelleklenmemesi gerektiği yazılmadıkça bir sonraki geliştirici uyarıyı geçerliliğini yitirmiş sayıp kaldırır.

Anlamanın ölçülmesi

Doğrudan ölçülemeyen bir değişken

“Kod anlaşıldı mı” sorusu doğrudan gözlemlenebilir bir büyüklüğe karşılık gelmez. Ölçülebilecek olan, anlamanın bıraktığı izlerdir: hangi davranışların belgelenmiş olduğu, bağımlılıkların ne kadarının bilindiği, kritik kararların gerekçelerinin kayıtlı olup olmadığı, üretimde çıkan olayların önceden bilinen sınırlar içinde kalıp kalmadığı.

Kapsam anlamanın ölçüsü değildir

Yaygın bir yanlış, test kapsamının anlaşılmanın göstergesi sayılmasıdır. Aşağıdaki örnek ayrımı görünür kılar:

```
def calculate_price(x):  
    return x * 1.18
```

Bu fonksiyon için yüzde yüz kapsam elde etmek kolaydır. Ancak katsayının neden 1.18 olduğu bilinmiyorsa, sistemin amaç ve evrim boyutları hakkında hiçbir bilgi yoktur. Kapsam, kodun çalıştırıldığını gösterir; denetlendiğini göstermez. Hiçbir doğrulama içermeyen bir test de kapsamı artırır.

Mutasyon testi

Kapsamın ölçemediği şeyi ölçen yöntem mutasyon testidir. Yöntem, koda küçük ve sistematik değişiklikler enjekte eder — bir karşılaştırmayı ters çevirir, bir koşulu kaldırır, bir sabiti değiştirir — ve testlerin bu değişiklikleri yakalayıp yakalamadığını ölçer. Yakalanmayan mutasyon, o bölgenin fiilen denetlenmediğini gösterir. Yöntem DeMillo, Lipton ve Sayward (1978) tarafından önerilmiş, Jia ve Harman (2011) tarafından derlenmiştir.

Böylece “sistem hakkındaki belirsizliği ne kadar azalttık” sorusuna sayısal bir yaklaşım elde edilir: mutasyon skoru, kapsamın aksine, testlerin gerçekten hata yakalama gücünü ölçer.

Belirsizlik azaltma çerçevesi

Beyaz sayfa benzetmesi burada bir hedef tanımına dönüşür. Ölçülmek istenen şey, çalıştırılan test sayısı değil, ortadan kaldırılan belirsizliktir. Bu çerçevenin bilgi kuramındaki karşılığı, bilginin belirsizlik azalması olarak tanımlanmasıdır (Shannon, 1948).

Çerçevenin pratik sınırı, boyutlar arası toplama yapılamamasıdır. Dört ya da beş boyutta ayrı ayrı yüksek skor elde edilmesi, bunların ortalamasının anlamayı temsil ettiği anlamına gelmez; boyutlar birbirinin yerine geçmez ve birinde eksik kalan bilgi diğerindeki fazlalıkla telafi edilmez.

Yerel çözümün küresel etkisi

Problem

Sık karşılaşılan bir durum şudur: bir hata düzeltilir, düzeltme beklenmedik bir yerde yeni bir hata üretir. Düzeltmeyi yapan kişi ya da araç, sorunu bulunduğu yerde ve en az değişiklikle çözmüştür. Yerel olarak doğru olan çözüm, sistemin başka bir yerindeki bir varsayımı bozmuştur.

Buradaki hata, düzeltmenin küçüklüğü değildir. Hata, değişikliğin hangi davranışlara dokunduğunun bilinmemesidir.

Değişiklik etki analizi

Bu sorunun literatürdeki karşılığı değişiklik etki analizidir. Amaç, bir değişikliğin doğrudan ve dolaylı olarak etkileyebileceği bileşenlerin kümesini önceden belirlemektir. Kavramın kökenindeki terim **dalgalanma etkisi**dir (*ripple effect*): bir noktadaki değişikliğin bağımlılık zinciri üzerinden uzak noktalara yayılması (Yau, Collofello ve MacGregor, 1978). Alanın standart derlemesi Bohner ve Arnold (1996) tarafından hazırlanmıştır.

İki klasik yaklaşımı vardır. Bağımlılık analizi, kod parçaları arasındaki çağrı ve veri bağımlılıklarını izler. İzlenebilirlik analizi ise gereksinim, tasarım, kod ve test arasındaki bağları izler.

Mevcut test kümesi bu analizde ikinci bir işlev üstlenir: değiştirilen kodun hangi davranışlarının başka testlerce korunduğunu gösterdiği için, kısmi bir bağımlılık haritası olarak okunabilir.

Etki analizinin sınırı

Etki analizi bir tahmindir. Statik bağımlılıklardan çıkarılamayan bağlar — paylaşılan durum, yapılandırma üzerinden kurulan bağımlılıklar, zamanlamaya bağlı etkileşimler, dolaylı yan etkiler — analizin dışında kalır. Beklenmedik yerden çıkan hata, tam olarak analizin göremediği bağdan gelir. Dolayısıyla etki analizi tek başına yeterli bir savunma değildir.

Değişmez denetimi

Etki analizinin kaçırdığını yakalayan mekanizma, konuma bağlı olmayan denetimdir. Bir **değişmez** (*invariant*), sistemde hangi değişiklik yapılırsa yapılsın doğru kalması gereken kuraldır. Sipariş toplamının kalemler toplamına eşit olması, ödeme tutarının sipariş tutarını aşmaması, indirim fiyatı geçmemesi bu türdendir. Kavramın yazılım tasarımındaki sistematik kullanımı sözleşmeye dayalı tasarım yaklaşımıyla kurulmuştur (Meyer, 1992).

Etki analiziyle arasındaki fark işlevseldir. Etki analizi “nereye bakmalıyım” sorusuna cevap verir ve cevabı eksik olabilir. Değişmez ise nereye bakıldığından bağımsızdır: kuralı bozan değişiklik hangi dosyada yapılırsa yapılsın yakalanır. Bu nedenle iki mekanizma birbirinin alternatifi değil, tamamlayıcıdır.

Benzer bir tamamlayıcılık, girdiler arası ilişkileri denetleyen testler için de geçerlidir. Bir hesaplamanın hangi kullanıcı için çalıştırıldığından bağımsız aynı sonucu vermesi, girdi sırasından etkilenmemesi, iki kez uygulandığında farklı sonuç üretmemesi — bunlar tek tek senaryolara değil, davranış sınıfına bağlanan denetimlerdir. Bir düzeltmenin uzaktaki bir varsayımı bozması, çoğunlukla bu tür bir sınıf ilişkisinin bozulması biçiminde ortaya çıkar.

Çalışma sırası ve ölçüt sorunu

Yaygın olarak önerilen sıra şudur: önce yerel problemi anla, sonra küresel etkiyi çıkar, sonra en basit güvenli çözümü seç, sonra doğrula.

Sıra makuldür ancak üçüncü adımdaki ölçüt tanımsızdır. “Güvenli” ölçütü verilmediğinde değerlendirme sezgiye kalır ve sezgi, tam olarak yerel çözüme kaymanın yeniden gerçekleşeceği yerdur. Ölçütün işlevsel hâli şudur: çözüm seçilmeden önce, değişikliğin bozabileceği değişmezler yazılır; seçim bu listeye karşı yapılır.

Bununla bağlantılı ikinci bir düzeltme, “en basit çözüm” ile “en küçük kod değişikliği”nin aynı şey sayılmasıdır. İki satırlık bir değişiklik, gizli bir varsayımı bozarak sekiz satırlık bir değişiklikten daha yüksek risk taşıyabilir. Küçültülmesi gereken büyüklük değişiklik boyutu değil, değişiklik sonrası sistem riskidir.

Sonuç

Test kümesini yalnızca doğrulama aracı olarak görmek, taşıdığı bilginin bir kısmını görünmez bırakır. Farklı test türleri farklı bilgi türleri kaydeder ve bu ayrım yapıldığında test kümesi, sistemin davranışsal kaydı olarak okunabilir hâle gelir. Bunun pratik sonucu, test yazma kararının “bu kod doğru mu” sorusuyla değil, “bu bilgi kaybolursa ne olur” sorusuyla verilmesidir.

Sistem anlamının boyutlarını yeniden adlandırmaya gerek yoktur. Zachman çerçevesi, mimari görünüm modelleri ve tasarım gerekçesi literatürü, aynı ayrımları halihazırda kurmuş durumdadır. Bu literatürlerin verdiği ek bilgi, boyut sayısının sabit olmadığıdır: görünümler paydaş ilgilerine göre belirlenir. Dolayısıyla “kaç boyut vardır” sorusu, cevabı olan bir soru değildir; anlamlı soru, belirli bir sistemde hangi bilgi türlerinin kayıt altına alınmadığıdır.

Bu boyutlar arasında amaç bilgisi ayrı bir konuma sahiptir, çünkü diğerlerinin aksine kodda ya da sürüm geçmişinde bulunmaz. Kaydedilmediğinde geri getirilemez ve kaybı ancak bir karar yanlışlıkla geri alındığında fark edilir. Kayıt maliyeti düşük, kayıp maliyeti yüksek olan bilgi türü budur.

Ölçüm tarafında çıkan sonuç, kapsamın yanlış büyüklüğü ölçtüğüdür. Kodun çalıştırılması ile denetlenmesi farklı şeylerdir ve aradaki farkı ölçen yöntem mutasyon testidir. Bununla birlikte hiçbir sayı “sistem anlaşıldı” önermesini doğrulamaz; ölçülen şey belirsizliğin azalmasıdır, anlamının kendisi değil.

Değişikliğin küresel etkisi konusunda çıkan sonuç ikilidir. Etki analizi gerekli fakat eksiktir; eksik kaldığı yeri kapatan şey, konumdan bağımsız çalışan değişmez denetimleridir. Bu ikisi birlikte kurulduğunda “bu düzeltme başka neyi bozar” sorusu, tahmine ek olarak bir yakalama mekanizmasına da bağlanmış olur.

Açık kalan iki soru vardır. Birincisi, tasarım gerekçesi kaydının hangi eşikten sonra maliyetli hâle geldiğidir: her karar için gerekçe yazmak sürdürülemez, hangi kararların bu eşiği aştığını belirleyen bir ölçüt bu notta kurulmamıştır. İkincisi, değişmezlerin keşfinin sistematikleştirilip sistematikleştirilemeyeceğidir; halihazırda hangi kuralın değişmez sayılacağı alan bilgisine bağlıdır ve eksik kalan değişmezleri ölçen bir yöntem bulunmamaktadır.

Kaynaklar

- Bohner, S. A., Arnold, R. S. (1996). *Software Change Impact Analysis*. IEEE Computer Society Press.
- Brooks, R. (1983). Towards a Theory of the Comprehension of Computer Programs. *International Journal of Man-Machine Studies*, 18(6).
- DeMillo, R. A., Lipton, R. J., Sayward, F. G. (1978). Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer*, 11(4).
- ISO/IEC/IEEE 42010:2011. *Systems and Software Engineering — Architecture Description*.
- Jia, Y., Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5).

- Kruchten, P. (1995). The 4+1 View Model of Architecture. *IEEE Software*, 12(6).
- Kunz, W., Rittel, H. W. J. (1970). *Issues as Elements of Information Systems*. Working Paper 131, University of California, Berkeley.
- Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. *Proceedings of the IEEE*, 68(9).
- Meyer, B. (1992). Applying Design by Contract. *IEEE Computer*, 25(10).
- Nygard, M. (2011). Documenting Architecture Decisions. *Cognitect Blog*.
- Parnas, D. L., Clements, P. C. (1986). A Rational Design Process: How and Why to Fake It. *IEEE Transactions on Software Engineering*, SE-12(2).
- Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27.
- von Mayrhauser, A., Vans, A. M. (1995). Program Comprehension During Software Maintenance and Evolution. *IEEE Computer*, 28(8).
- Yau, S. S., Collofello, J. S., MacGregor, T. (1978). Ripple Effect Analysis of Software Maintenance. *COMPSAC*.
- Zachman, J. A. (1987). A Framework for Information Systems Architecture. *IBM Systems Journal*, 26(3).